4 Distributed file systems

Contents

4.1 Introduction

4.1.1 Properties of distributed file systems

4.1.2 Characterization of the use of files

4.2 System model

4.2.1 Structure

4.2.2 Identification of files

4.3 Name service

4.4 File service

4.4.1 Semantics of sharing

4.4.2 Types of servers

4.4.3 Caching and consistency management

4.5 Examples

4.5.1 Network File System (NFS)

4.5.2 Andrew File System (AFS)

4.6 Exercises

4.1 Introduction

File systems and databases are perhaps the first resources to be distributed and shared in a networked system. In fact, there are commercial products for a long time for local area networks, as it is the case of NFS. More recently, distributed file systems for wide area networks have also been developed.

The management of a distributed file system is supported by two often well differentiated functions: a **name or directory service** and a **file service**. It is essential to provide an acceptable performance, so we must reach a compromise between the local availability of information to reduce communication costs (***caching*** and **distribution**) and consistency, which is reflected in the semantics shown by shared access.

4.1.1 Properties of distributed file systems

A file system is characterized by a set of general properties:

- it provides permanent information storage;
- it identifies the files in a namespace (normally structured);
- concurrent access is possible from several processes;
- in multi-user systems it provides access protection.

A distributed file system also has the following properties as objectives:

- Transparency in file identification. Unique namespace, independent from the client.
- Transparency in file location. To allow the mobility of files from one location to another, a dynamic mapping name-location is required.
- Scalability. Structured namespaces, and replication (caching) to avoid bottlenecks.
- Robustness to failures. The server should not be affected by client failures, in which concerns the management of the state of clients on the server. On the other hand, the interface offered to the clients must provide as far as

possible *idempotent*[1] operations, which guarantee the correctness of repeated requests (e.g., due to failure suspicion) to the server.

- High availability and fault tolerance. They involve some form of replication. One aspect of availability is to allow operation in *disconnected mode*, which requires the caching of entire files.

- Consistency. The objective is to provide the semantics of centralized systems as much as possible, for example, to preserve Unix semantics in the presence of caching or other forms of replication.

- Security. The need for remote authentication implies new protection models, based on credentials instead of access lists.

4.1.2 Characterization of the use of files

The design of a distributed file system that provides a good level of performance should be based on the characteristics of use of the files by the applications. Although it is not easy to generalize, it is possible to determine a series of patterns of behavior:

- Most files are small. This implies that the file can be the access unit.

- Writing is infrequent. This encourages caching and replication.

- Sharing (concurrent access) is infrequent. Most files are accessed by a reader and/or a writer, some files are accessed by *n* readers and a writer, and they are very rarely accessed by *n* readers and *m* writers[2]. Therefore, an *optimistic* management of caching and replication can be profitable, which presupposes that there is a single writer and rectifies in case of subsequent detection of simultaneous writings.

- The search/use ratio is usually low[3]. This fact also encourages caching.

- The access is usually sequential and there is a high degree of locality. This encourages *buffering* to provide anticipation in accesses.

---

[1] A function *f* is idempotent when $f(f(x)) = f(x)$.

[2] In fact not even Unix semantics in centralized systems manages the synchronization between several writers, passing the responsibility to the applications.

[3] This relation is a measure of the access rate to the file in proportion to the amount of processing that is performed on the accessed element. In databases this ratio is high because applications usually access a large number of elements and in most cases for a simple query. On the other hand, in the case of accesses to files it is usual to carry out a more or less expensive processing for each element accessed.

- Most files have a very short life (for example, temporary files). We must tend to manage them locally.
- There are classes of files with differentiated properties (for example, executable files, which are rarely modified).

4.2 System model

The distributed file system model that we are going to define here [COU05 §8] considers a client-server structure that includes differentiated name and file services, and a mechanism for unique identification of the files by the clients.

4.2.1 Structure

The structure of the distributed file system model that we are presenting consists of three modules:

Client

It is the local interface with the application. It interprets system calls on files and generates requests (usually remote procedure calls, RPCs) for remote access. It knows the location of the name and file services, and manages local storage (caching).

File service

It maintains the contents of the files (and directories) and the attributes of the files: creation, last access and last modification times, length, number of references… A file is identified in the file service by a *unique file identifier*, UFID. The operations on a file refer explicitly to its UFID. The file service interface with the client offers operations such as *read*, *write*, *create*, *delete*, *get_attributes* and *set_attributes*.

Name (directory) service

It is in charge of providing location transparency. In general, it is a database with (name, UFID) elements, where entries are created, modified and searched. The name is specified by the character string that describes the path. The name service interface with the client offers operations such as *search_name*, *add_name* and *delete_name*. Some attributes of the file are maintained by the name service: file type (ordinary or directory), user identifier of the owner of the file, access rights…

4.2.2 Identification of files

The client specifies a file to the file server by means of the unique file identifier, UFID, issued by the file server when a file is created. In principle, the UFID requires identifying:

In principle, the UFID requires identifying:

- The file server, *S*.
- The file inside the server, *F*.
- The access rights over the file, *D*.

An objective of the UFID is to guarantee the correct use of the access rights of the client on the file, preventing the manipulation of the UFID (e.g., to claim access rights that the client does not have). In a Unix centralized file system, the process that accesses the file has a user identifier associated with it, which is compared with the one stored in the inode of the file to authenticate the client[4], allowing the permissions defined in the inode to be applied. In contrast, in a distributed file system there is no implicit user identification, so ensuring the secure application of access rights requires an additional mechanism. A model is the one that we will describe next, analogous to that of Amoeba [TAN95 §7].

The UFID is a credential (*capability*) for accessing the file. When a file is created, the file server generates a random number, *R*, as an additional component of the UFID of the file that is stored associated with the name of the file for use by the name server. The UFID issued by the name server for a file requested by a client (in a *search_name* operation) includes a *C* field that is an encoding of *R* and the access rights for that client (these are also kept uncoded in a field of the UFID, *D*, for the use of the client). The coding is done using a unidirectional function, f_c:

$$C = f_c\ (R, D)$$

In the operations on the file, to check the validity of an access, the file server will apply the function f_c on the stored *R* value and the *D* field of the client's UFID, and compare the result with the *C* field of the client's UFID:

$$f_c\ (R, D) = C\ ?$$

A client claiming access rights *D'* other than the ones it really has will not pass this check. The scheme of this UFID model is shown in Figure 1.

[4] In Unix, the user (UID) and group (GID) comparison is applied.

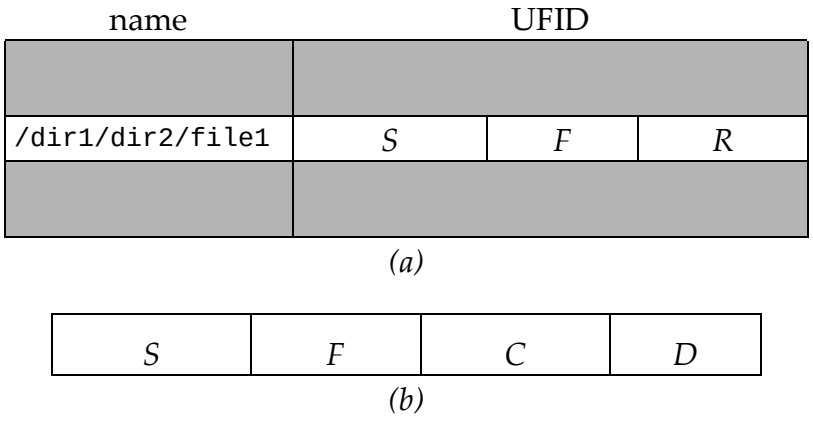


Figure 1. The UFID of a file `/dir1/dir2/file1` *(a) as stored in the name server, (b) as specified in the request of a client.*

4.3 Name service

We require a service that allows the client module to determine the location of a file (more specifically, the file UFID) from the symbolic name of the file (path). It is strictly a directory service, although, by generality, this type of service is usually called a name service, and is also used for other types of distributed applications.

As known, directories have a tree structure. From the point of view of name resolution, the structure is divided into **domains**, so that a domain can be associated with a part of the path. A name server manages one or more domains. The client module maintains a table of entries (domain, server), so that, given a path, it searches the table by comparing strings to which domain it belongs.

A name server can resolve the path as associated to a domain maintained by another name server[5], which occurs in particular in wide area networks. In this case, the resolution of the symbolic name is indirect and will chain a succession of requests to different servers, which in general is known as **navigation**.

We can distinguish different navigation schemes:

- **Iterative**. The server of a domain resolves its part of the path and responds to the client by indicating the new server to resolve the rest of the path. The communication is done by conventional RPCs. This alternative is limited to a single *administrative domain*, since the client may not have access outside of

[5] In general, unknown by the client, since otherwise it is expected that the association was registered locally.

it. An example is the NIS[6] server, used by NFS. A variant is the **iterative navigation controlled by the server**. In this case, the server invoked by the client is responsible for performing (iteratively) the sequence of requests for the resolution of the path, responding to the client when it has been completely resolved

- **Recursive**. The server of a domain resolves its part of the path and sends a request to a new server to resolve the rest of the path. The server that resolves the last element of the path is the one that responds to the client, so this scheme does not allow using conventional RPCs[7]. On the contrary, it requires less communication than the previous scheme. An example is the DNS server, which is used on the Internet[8].

To reduce latencies, name services make extensive use of caching, which may lead to inconsistent situations between name servers. However, since the migration of domains between servers is infrequent, it is not usually a priority objective of the name service to prevent possible situations of erroneous identification of domains due to caching.

4.4 File service

The objective in the design of the distributed file servers is to provide a semantics as close as possible to the one offered by the centralized file systems without incurring a heavy penalty in the performance (fundamentally the latency). Caching is widely used and copy management mechanisms are used that allow for reasonable compromises between semantics and performance.

4.4.1 Semantics of sharing

As we have seen, it is necessary to establish a compromise between the desired performance and the semantics that we want to guarantee in the distributed file system when there are concurrent accesses. We can distinguish several types of semantics:

- **Unix semantics**. So called because it is what characterizes access to the files of classic Unix file systems. The operations on a file are completely ordered

[6] Although it must be emphasized that NIS is not a directory service that fits the defined model, since it does not manage UFIDs, but a more general name service that resolves navigation.
[7] In this case (direct response to the client), the search is said to be transitive. An alternative with greater latency is that the return follows the inverse path node to node, which has the advantage that the nodes can cache the resolved names.
[8] DNS also supports iterative navigation.

in time, which implies that a reading returns the last update of the file. As we will see, it is complex for a distributed file system to provide Unix semantics.

- **Session semantics**. In a session of use of a file (since it is opened until it is closed) the process sees a private copy of the file, that is, the state of the file is not shared (for example, the pointer to the current position on the file). In a distributed file system session semantics is equivalent to working on a local copy that is loaded when the file is opened and updated on the server when the file is closed. Race conditions may occur when the updated copy is written to the server, whose resolution is the responsibility of the user or the application.

- **Immutable files**. Files are not modified, but they are replaced atomically (directories are thus modified) by new versions. It is possible the concurrent sharing for reading, but if a process wants to access a file opened by another process for writing, there are two alternatives: (a) consider the opening operation as an error, and (b) obtain the previous version of the file. This semantics is appropriate for particular services, such as backup services and information repositories with version management (for example, *subversion*[9]).

- **Transaction semantics**. As we saw in the previous chapter, the use of transactions allows explicitly defining sequences of operations on files, guaranteeing the transactional semantics defined by the *ACID* properties.

4.4.2 Types of servers

Depending on the information stored by the server about the file that is being accessed by a client, two categories of servers can be distinguished, which will determine the characteristics of the service provided.

- **Server without state**. The server does not store client information in a session on a file. For this reason, the client-server interface offered by this type of server do not include specific open/close operations. The client provides in each request all the necessary information to perform the operation, including the pointer to the current access position on the file. One of the main advantages of this approach is that the failure of a client does not affect the server at all. On the other hand, it makes it difficult to provide Unix semantics (for example, access rights to the file are checked

[9] http://subversion.apache.org/

at each request). The most notable example of a stateless server is NFS (up to Version 3).

- **Server with state**. The server creates an entry for the file upon a request to open a client's file. The successive requests of access to the file require shorter messages, since the server stores the pointer to the last accessed position on the file and other information, which may even include blocks of the file, which enables anticipated reading in the server. This approach inherently limits the number of open files simultaneously. The main drawback is that the server has to manage the possible failure of the clients, in order to release the state information of the files opened by the client that fails, thus preventing the degradation of the service. An example of a server with state is RFS (*Remote File System*), currently very little used.

By its very nature, the operations of the client-server interface in a stateless distributed file system, having to specify in the request all the parameters of the operation, tend to be idempotent, which allows a simpler management of the reinvocations in case of suspicion of failure in the transmission of the request. For example, for a server with state a read operation would be specified as:

```
result= read_file(ufid, buffer, length)
```

while the analogous operation for a stateless server would be:

```
result= read_file(ufid, buffer, position, length)
```

The `position` parameter is a pointer to the next byte to read in the file. Note that in a server with state this parameter is managed by the server through the table of open files, where an entry has been reserved as a consequence of the previous operation of opening the file. This entry will include the pointer to the position of the file that the server will update after each access. On the other hand, a stateless server does not manage the files opened by the client, so the client must manage the access position and specify it in each request. If, after the timeout of a response expires, the client forwards the invocation with the same parameters, the behavior differs in both types of servers. In a stateless server the operation will always behave as idempotent because the server, in each reading, will open the file, will position at `position` and will perform the access. However, on a server with state, if the first request message ended up arriving at the server, it will have updated the current access pointer in the table of open files, so the reinvocation will be treated as a new operation.

4.4.3 Caching and consistency management

Caching refers to the temporary storage of (pieces of) files in the client node[10], in order to minimize the communication costs associated with remote access.

To do so, local copies of (part of) the files are maintained in the client nodes, using the temporal locality as the space management criterion. The management unit, i.e., the amount of information that is transmitted in each request, can be the complete file or a block[11]. In fact, transmitting large amounts of information provides **anticipated reading** (*buffering*), which also enhances the spatial locality.

Caching can be supported either on the local disk, **persistently**, or in memory, **not persistent**. In the latter case, kernel memory can be used, which optimizes storage space, or user memory. Microkernels, like Mach 3.0, allow the use of a memory manager (defined outside the microkernel, in user space) that maps the blocks stored in their address space in the address spaces of the processes that request said blocks, so that there is a single copy of each block and in turn is visible in the address space of each process that uses it.

A generalization of temporary storage, in wide area networks, is **structured caching**, i.e., the temporary storage in intermediate nodes at different levels. This service is offered by the *proxy* nodes on the Internet.

Caching, like all forms of replication, is associated with the need to manage consistency. The policy of consistency management conditions the semantics of sharing. The basic policies to manage the consistency are the following:

- **Write-through**. When a block/file is modified, it is copied to the server. Adaptations are usually introduced to improve performance, such as **delayed writing** (accumulating modifications), and not to copy temporary files to the server.

- **Write-on-close**. The file is written to the server when it is closed. This policy provides session semantics. An improvement consists in delaying the writing (it is common for a file to be deleted after closing).

- **Centralized management**. The file server manages the opening/closing of files using a readers-writers synchronization algorithm. This policy

[10] The server can also provide *buffer cache* of blocks as in any centralized file system.

[11] The size of the block used as the caching management unit referred to here does not have to match the block size defined for the local file system of each node.

provides Unix semantics, but it is not very scalable and does not tolerate failures.

Except in the case of the centralized management, the consistency management policies described are not sufficient to provide the defined semantics (in particular, the Unix semantics), requiring an additional **validation** mechanism to allow a node to know when the replica of a file that is being used in its cache has become obsolete by the update of another replica of the same file in another node (not necessarily the server). There are two basic validation policies, depending on where the initiative comes from:

(a) From the clients, by periodically accessing the attributes of the file on the server, to see if they have been modified. The period between validations is a critical parameter: preserving semantics requires short periods, at the cost of overloading the network.

(b) From the server, by notifying clients when a copy has become obsolete (***callback***). It requires storing some state on the server.

4.5 Examples

4.5.1 Network File System (NFS)

Introduced by Sun Microsystems in the 1980s, it was originally developed for Unix. It was conceived as an open system, which has allowed it to be adopted by all Unix families and by other operating systems (VMS, Windows, MacOS), becoming a de facto standard in local area networks. NFS has evolved a lot and the current Version 4 has little to do with the previous ones, since it includes state in the server and the possibility of deployment in wide area networks. However, here we will refer to the characteristics of the previous versions. A general description of NFS can be found in [TAN95 §5] [VAH96 §10] [BRO94 §3] [COU05 §8].

4.5.1.1 General characteristics

Servers *export* directories. To make a directory exportable, the path is included in a specific configuration file. Clients *mount* the exported directories, and these are seen in the client completely integrated into the file system. Mounting is executed in the booting of the operating system, or on demand when a file is opened through an additional NFS service, the *automounter*. The operations on files and the mounting requests are handled by separate daemon processes on the server (*nfsd* and *mountd* respectively).

NFS servers are stateless, which avoids having to deal with client failures on the server. Thanks to the fact that most of the operations are idempotent, the management of communication errors in the client is simplified.

The semantics of sharing tries to be Unix, although with some limitation, due mainly to the management of the caching and to its condition as a stateless server. It offers the same Unix protection model, although, due to the absence of state in the server, the access rights are checked in each access operation to the file instead of only when opening. Initially NFS did not adopt any authentication mechanism. The client interface included in the RPCs the Unix user identifier, which was checked on the server, which did not prevent the possibility of supplanting the identity of a user by building an RPC outside the one offered by the interface. Currently NFS is often combined with authentication systems such as Kerberos.

NFS uses the NIS (*Network Information System*) service to centralize information about the location of servers.

4.5.1.2 Interfaces

The applications use the Unix interface. NFS defines an interface for client-server communication, consisting of three protocols:

- The RPC protocol defines the model of client-server communication. The data is serialized according to the XDR format. The communication is based on UDP. As of Version 3, TCP is also supported for wide area network communication.
- Protocol for mount/unmount directory operations [VAH96 §10].
- NFS protocol. Procedures for file operations (search, create, read, write, delete, obtain attributes…). Consult [VAH96 §10] [COU05, Fig. 8.9] for a description of the protocol.

4.5.1.3 Implementation

Identification of files

The clients specify as file's unique identifier a *file handle*, provided by the *lookup* operation (search of the file name). It is a data structure that contains information for the identification of the file on the server, mainly the file system identifier and the inode number. The file handle is opaque for the client (it does not manage its content). The resolution of the path is done iteratively in the client, requiring a lookup operation for each component of the path.

Virtual file system, VFS

On Unix, NFS uses the *Virtual File System (VFS)*[12] interface for a transparent access to local and remote files. VFS manages a *v-node* (virtual inode) for each open file. If the file it represents is local, the v-node points directly to the corresponding inode in the local file system. If the file is remote, the v-node points to an *r-node* (remote inode) that stores the file handle that the NFS client will use in the RPCs. Figure 2 represents this architecture.

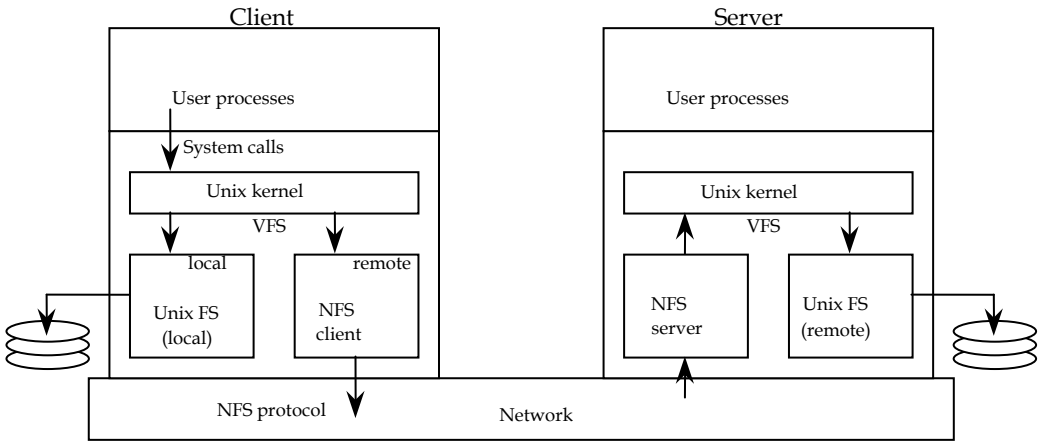


Figure 2. Architecture of NFS

Caching

NFS works with large blocks (typically 16 Kbytes), which provides anticipated reading. The writing policy is delayed *write-through*. Only complete blocks are sent to the server.

Periodically the client validates the blocks in its cache by checking in the server if the attributes have changed (through an RPC to obtain attributes, *getattr*) and updates the copy if it is the case. The validation period is usually 30 seconds.

4.5.1.4 Limitations of NFS

Providing Unix semantics in NFS is problematic. The reduction of the cache validation period to improve the consistency produces an overload due to the large number of *getattr* operations that are performed.

[12] VFS provides Unix applications with an independent file system interface, which allows supporting both NFS and any other file system. See, for example, [VAH96 §8].

In principle, the mounting of remote file systems was not transparent (the server must be identified). The *automounter*, a utility that allows the dynamic mounting of file systems on demand, improves this aspect.

Due to the lack of state in the server, blocking access to remote files requires an independent mutual exclusion mechanism. In Unix, a specific server, *lockd*, is used.

NFS is not designed to support server replication. To increase availability, parts of the file system that have to support a very high rate of access can be replicated in a set of servers, provided they are for reading. This is done with the NIS, so that each replica is accessible for reading, but the writing is always done on the master copy and the replicas are updated manually.

In principle, NFS was conceived for local area networks of a few dozen nodes, although improvements in LAN technologies and optimizations introduced in the latest versions of NFS allow supporting a much larger number of clients. As of Version 3, it allows configurations in wide area networks.

4.5.2 Andrew File System (AFS)

Andrew is the name of a family of distributed file systems for Unix developed at the Carnegie Mellon University in the 1980s. The components of the family are:

- AFS-1 (1983). Not optimized prototype.
- AFS-2 (1985)
- AFS-3 (1988)
- Coda (1987). Provides operation in disconnected mode.

AFS can be defined in general as a distributed file system with state that provides session semantics. Here we will present the general characteristics. For more details, the following references can be consulted: [MUL93 §14] [TAN93 §13] [COU05 §8] [SAT90].

4.5.2.1 Architecture

The architecture of AFS consists of two components, one on the server and one on the client:

- *Vice*: Server side. From the point of view of the client, Vice is a set of file servers interconnected in a network.

- *Venus*: Client side that runs on the operating system on the nodes connected to Vice.

Vice files are seen as integrated into the local file system of each client node.

AFS supports replication of subsets of the file system (*volumes*) of infrequent update (AFS-2). This technique is also used for backups (by means of read-only copies of a *volume*).

4.5.2.2 Implementation

AFS-1, AFS-2 and Coda work with entire files; AFS-3 works with blocks of 64 Kbytes. Caching is implemented on the client's disk.

The cache consistency policy is *write-on-close*. Session semantics is provided by *callbacks* (as of AFS-2). When Vice sends a file to a client, it also provides a *callback promise*[13]. When a client closes a modified file, Vice communicates to the clients for whom it keeps a callback promise of that file in order to cancel the promise. The Venus module of a client accessing the local copy of a file with the canceled promise will be responsible for reloading the new version of the file from the Vice server.

AFS-3 introduces important optimizations with respect to AFS-2. It inserts Venus in the kernel (using the VFS interface, as in NFS) and defines server *cells* to scale the system to wide area networks.

4.5.2.3 Security

AFS defines access domains like Unix. The access rights are defined in a way compatible with Unix.

It provides authentication through an authentication server that issues *tokens* upon presentation of the user's password in the login to access the file system for a pre-established period (typically 24 hours). Some version of AFS-3 have adopted Kerberos as authentication service.

4.5.2.4 High availability: Coda

Coda is an AFS version designed to provide high availability in environments subject to failures, both in the network and in the servers, so it is suitable for

[13] This is the only information about the client state that AFS manages on the server.

mobile devices (subject to frequent disconnections) and in general in replicated systems that require fault tolerance.

It manages replicas of *volumes* following an optimistic strategy. For this, it is based on two mechanisms:

- It uses version numbers and time vectors for the consistent update of the replicas (sometimes it requires manual intervention).
- It operates on the local cache when it loses connection with the server until it manages to connect to another server (operation in disconnected mode).

The basic characteristics of Coda are those of AFS-2. For more information, see [COU05 §15] and [SAT90].

4.6 Exercises

1 Two processes redirect their outputs to a common file *x.out*. Both processes have opened the file in *append* mode. One process writes 10,000 ASCII bytes '*a*' and the other 10,000 '*b*'. Explain the differences in behavior according to if the distributed file system provides Unix semantics or session semantics. What are the possible outputs in each case?

2 From the programmer's manual of a distributed file system we have extracted the following RPC specifications for the file server:

```
result= read_remote_file(ufid, buffer, length)
```

Reads *length* bytes from the file identified by *ufid* into *buffer*. Returns the number of bytes read or *ERROR*.

```
result= write_remote_file(ufid, buffer, length)
```

Writes *length* bytes from *buffer* in the file identified by *ufid*. Returns the number of bytes written or *ERROR*.

(a) Discuss if it is a file server with state or without state.

(b) What can be said about the idempotency or not of these operations?

3 Given the list of RPCs for NFS client-server communication ([COU05 Fig. 8.9]), which are idempotent and which are not?

4 Explain the effect of the consistency management in NFS (write-through policy and periodic verification of the validity of files), as well as delayed writing. How does the validation period affect Unix semantics?

5 In a stateless distributed file system (such as NFS), a *unlink* system call may not respect Unix semantics. Explain why. Hint: when the Unix *unlink* causes the reference counter of the file to reach zero, there may be processes still accessing the file, so Unix does not release the inode until the last of the processes closes the file.

6 The Unix file system allows a process to block access to a file (flag O_EXCL in *open*). Discuss if NFS can provide this semantics.